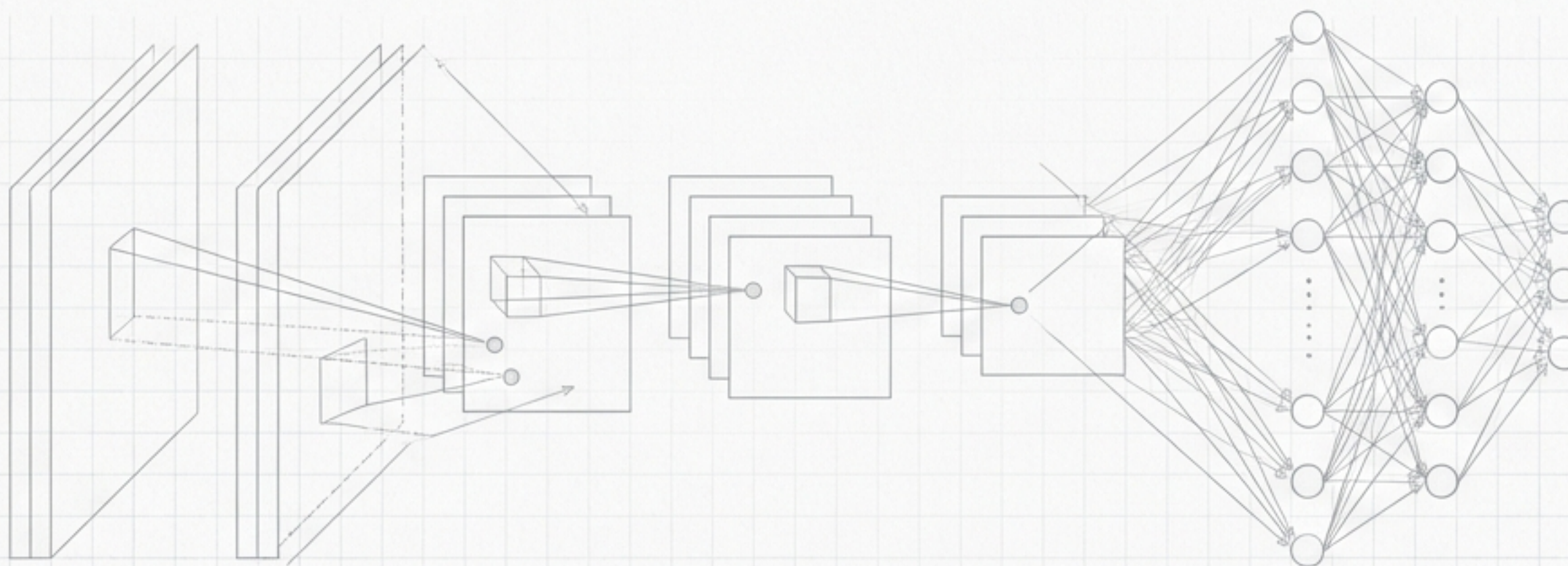


Chương 6. Mạng CNN (Tiếp theo)

Tiết 2/3 - Pretrained Models và Học chuyển giao (Transfer Learning)

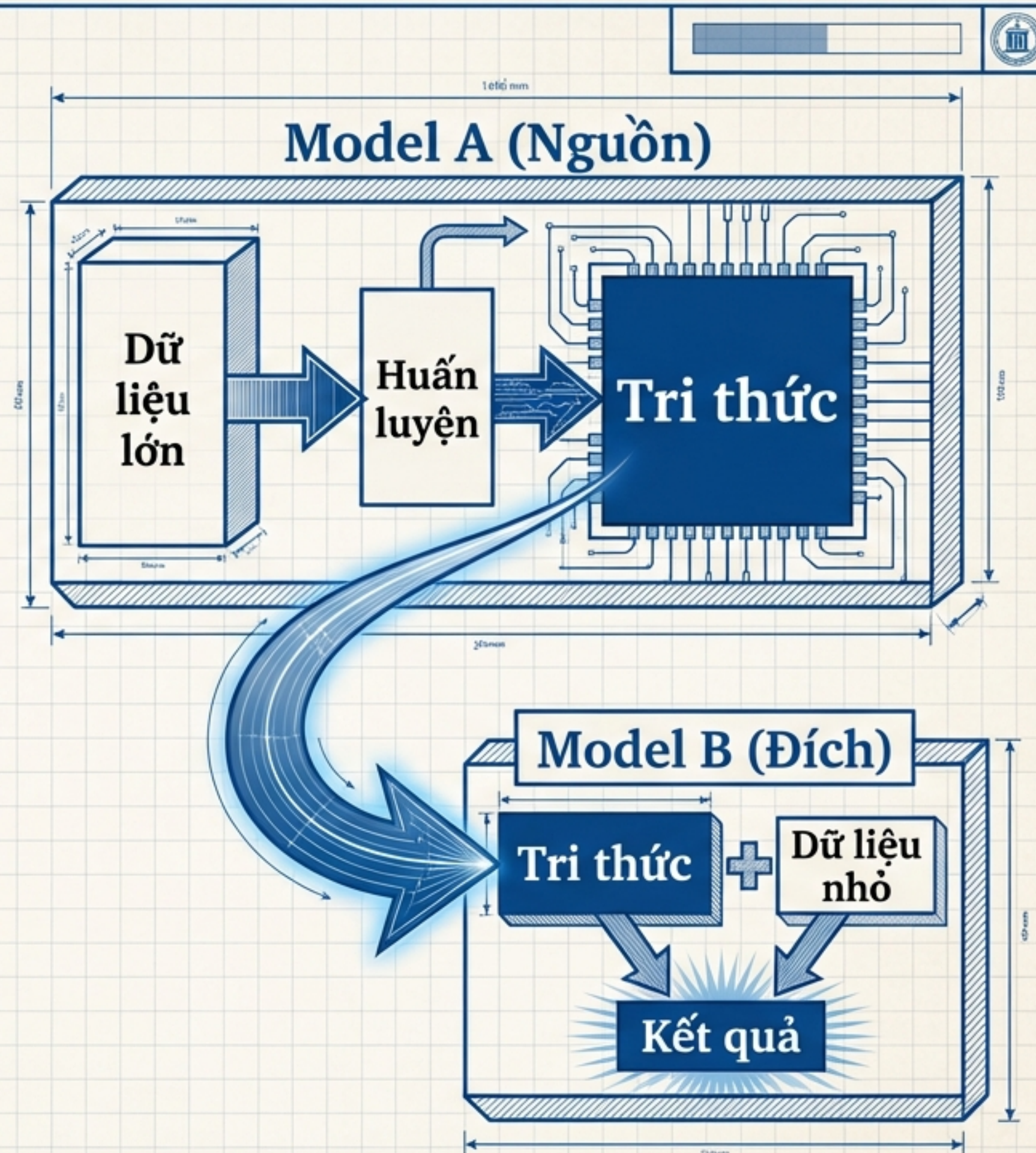


6.3.1 Khái niệm Học chuyển giao (Transfer Learning)

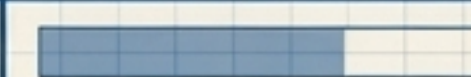
Định nghĩa: Chuyển giao tri thức từ một bài toán đã giải quyết sang một bài toán mới.

Cách tiếp cận cốt lõi: Thay vì huấn luyện một mạng nơ-ron sâu (DNN) khổng lồ từ con số 0, hãy tái sử dụng một mạng hiện có từ một tác vụ tương tự.

Đặc thù với CNN: Hoạt động cực kỳ hiệu quả với Mạng nơ-ron tích chập (CNN) vì các lớp thấp của CNN luôn học được các đặc trưng mang tính tổng quát cao.



Tại sao cần sử dụng Transfer Learning?



Tăng tốc độ hội tụ

- Khởi tạo bằng trọng số đã học thay vì ngẫu nhiên.
- Bỏ qua việc học lại các cấu trúc cơ bản (góc, cạnh).
- Rút ngắn đáng kể thời gian huấn luyện.



Tiết kiệm dữ liệu gán nhãn

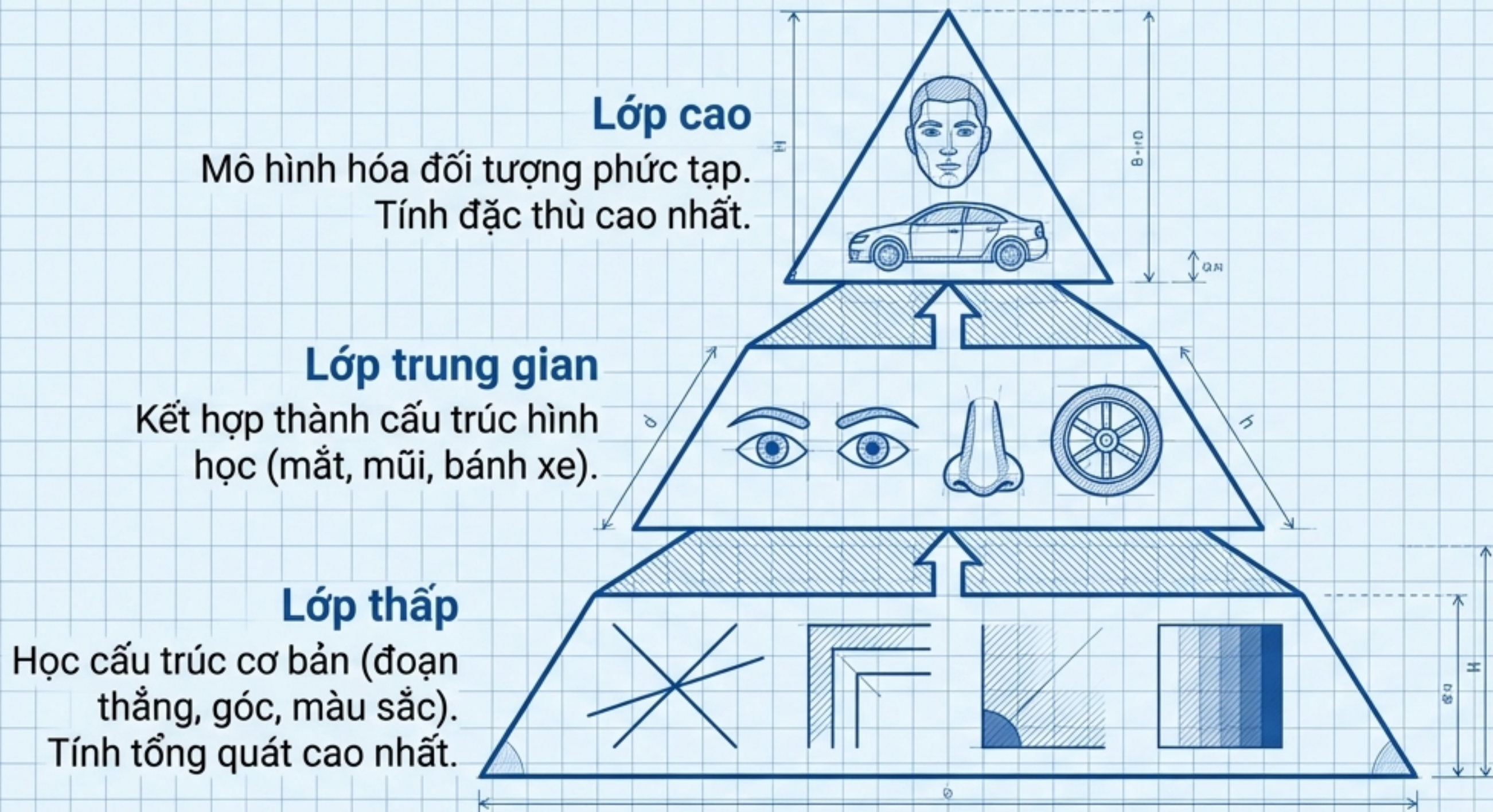
- Giải quyết bài toán phức tạp ngay cả khi có rất ít dữ liệu.
- Khắc phục rào cản chi phí gán nhãn dữ liệu đắt đỏ.



Cải thiện hiệu suất

- Tái sử dụng kiến trúc phân cấp đã được tối ưu hóa.
- Tăng cường khả năng tổng quát hóa (generalization) trên tập dữ liệu mới.

6.3.2 Cơ chế tái sử dụng các lớp (Reusing Pretrained Layers)



Cơ chế: Tái sử dụng phần lớn các lớp từ mạng đã huấn luyện (VD: phân loại 100 loại động vật), chỉ loại bỏ các lớp trên cùng. Hiệu quả nhất khi bài toán mới có chung đặc trưng cấp thấp với bài toán gốc.

Chiến lược 1: Đóng băng các lớp (Feature Extraction)



2. Huấn luyện lớp Output mới:

Thêm lớp đầu ra mới và chỉ huấn luyện duy nhất lớp này.

1. Đóng băng trọng số (Freeze):

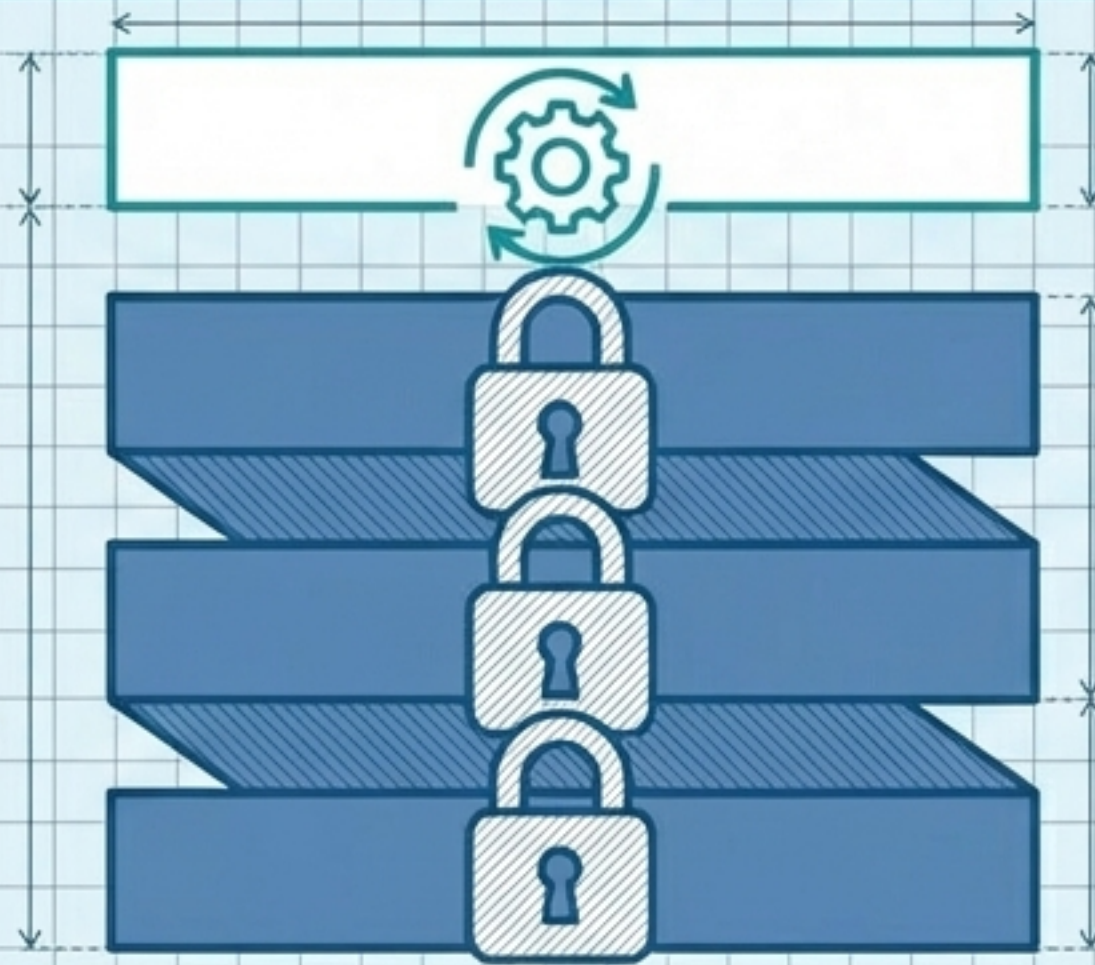
Thiết lập các lớp tái sử dụng ở trạng thái không thể huấn luyện (`trainable = False`). Gradient descent sẽ không làm thay đổi các trọng số tinh tế này.

Nguyên tắc vàng: Hai bài toán càng giống nhau -> Càng nên tái sử dụng nhiều lớp.
Nếu cực kỳ giống nhau: Giữ lại toàn bộ lớp ẩn, chỉ thay lớp Output.

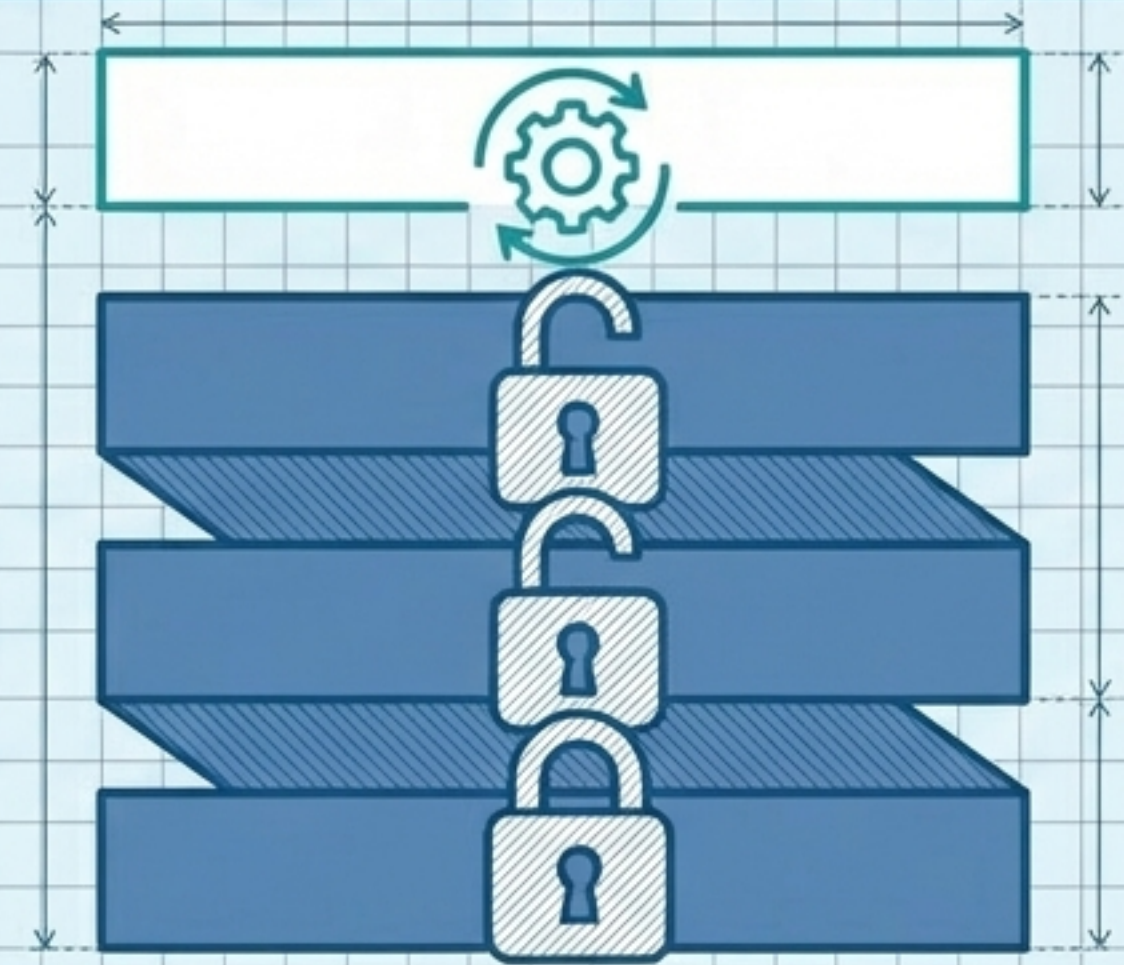
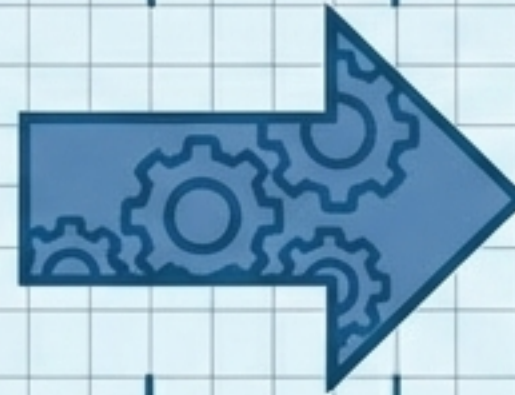
Chiến lược 2: Tinh chỉnh (Fine-Tuning)

Khái niệm: Mở băng (Unfreeze) một số lớp ẩn ở tầng trên cùng của mô hình Pretrained để huấn luyện tiếp cùng với lớp Output mới.

Mục đích: Giúp các đặc trưng cấp cao tự điều chỉnh sát hơn với đặc thù dữ liệu mới.



Bước 1: Đóng băng toàn bộ lớp cũ. Huấn luyện lớp Output mới trong vài epoch để ổn định trọng số cơ bản.



Bước 2: Mở băng các lớp ẩn trên cùng. Tiếp tục huấn luyện toàn bộ mô hình (với learning rate rất nhỏ).

Lưu ý quan trọng khi Tinh chỉnh (Fine-Tuning)

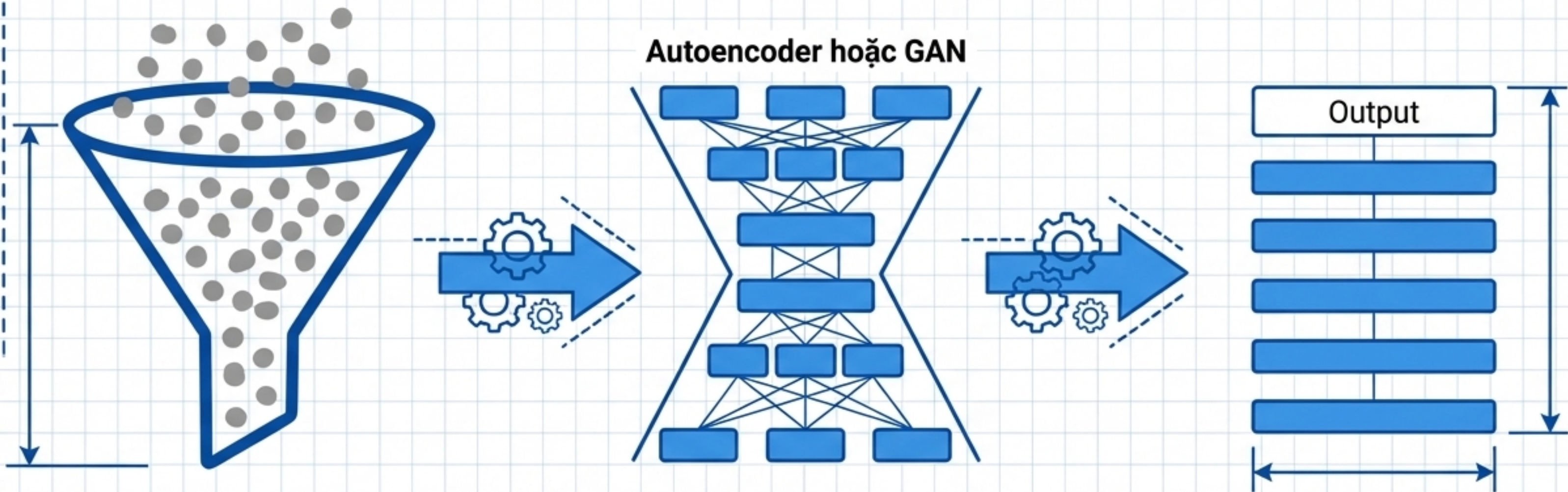


Cảnh báo Tốc độ học (Learning Rate):
Bắt buộc phải giảm tốc độ học (nhỏ hơn hoặc bằng 10 lần) khi mở băng. Tốc độ học lớn sẽ phá hủy các trọng số đã được tối ưu từ trước.

	Dữ liệu Huấn luyện ÍT	Dữ liệu Huấn luyện NHIỀU
Chiến lược	Chỉ tinh chỉnh lớp phân loại cuối hoặc 1 lớp ẩn trên cùng.	Mở băng nhiều lớp ẩn hơn.
Lý do	Ngăn chặn quá khớp (Overfitting) nghiêm trọng.	Cho phép mô hình học các đặc trưng hoàn toàn mới sát với bài toán.

Chiến lược 3: Tiền huấn luyện không giám sát (Unsupervised Pretraining)

Vấn đề: Bài toán phức tạp, rất ít dữ liệu gán nhãn, và không có mô hình pretrained tương đương.



Bước 1: Thu thập

Lấy lượng lớn dữ liệu không gán nhãn (Unlabeled data - rẻ & dễ kiếm).

Bước 2: Huấn luyện không giám sát

Dùng dữ liệu này đào tạo một mô hình Autoencoder hoặc GAN. Mô hình tự tìm ra cấu trúc ẩn.

Bước 3: Học chuyển giao

Tái sử dụng các lớp thấp của Autoencoder/GAN, thêm lớp Output, và Fine-tune bằng lượng nhỏ dữ liệu có gán nhãn.

Chiến lược 4: Tiền huấn luyện trên Tác vụ phụ (Auxiliary Task)

Khái niệm: Huấn luyện mạng đầu tiên trên một tác vụ phụ (dễ tạo ra nhãn hàng loạt). Sau đó, dùng các lớp trích xuất đặc trưng cho tác vụ chính.

Tác vụ phụ
(Dữ liệu khổng lồ):



Tải hàng triệu ảnh ngẫu nhiên trên web. Huấn luyện mạng (Siamese Network) trả lời câu hỏi: Hai ảnh này có phải cùng một người không?

Siamese Network



Kết quả: Tái sử dụng bộ trích xuất đặc trưng xuất sắc từ tác vụ phụ cho hệ thống của công ty.

Tác vụ thực
(Ít dữ liệu):



Nhận diện nhân viên công ty (chỉ có 2-3 ảnh/người).
-> Không đủ để huấn luyện DNN.



Giới thiệu sơ lược các Pretrained Models phổ biến

Mô hình cơ sở thường được huấn luyện trên các tập dữ liệu khổng lồ
(VD: ImageNet - 1.2 triệu ảnh, 1000 lớp).

(Nền tảng cho Chương 7)

VGG (16/19)

Kiến trúc kinh điển, cấu trúc đơn giản, mạch thẳng. Dễ hiểu nhưng dung lượng lớn, tính toán nặng.

ResNet (Residual Networks)

Cột mốc lịch sử. Dùng kết nối tắt (skip connections) giải quyết triệt tiêu đạo hàm, cho phép mạng sâu tới hàng trăm lớp.

Inception (GoogLeNet)

Mở rộng chiều ngang, dùng các khối tính toán song song đa tỷ lệ để nắm bắt đặc trưng ở nhiều kích cỡ.

MobileNet / EfficientNet

Tối ưu hóa dung lượng & FLOPs. Cân bằng hoàn hảo giữa độ chính xác và tốc độ cho thiết bị di động/IoT.

Thực hành: Tải mô hình cơ sở (Base Model)

Keras (TensorFlow) hoặc **TorchVision (PyTorch)** cho phép tải mô hình chỉ với 1 dòng lệnh.

```
base_model = tf.keras.applications.Xception.Xception(  
    weights='imagenet',  
    include_top=False  
)
```

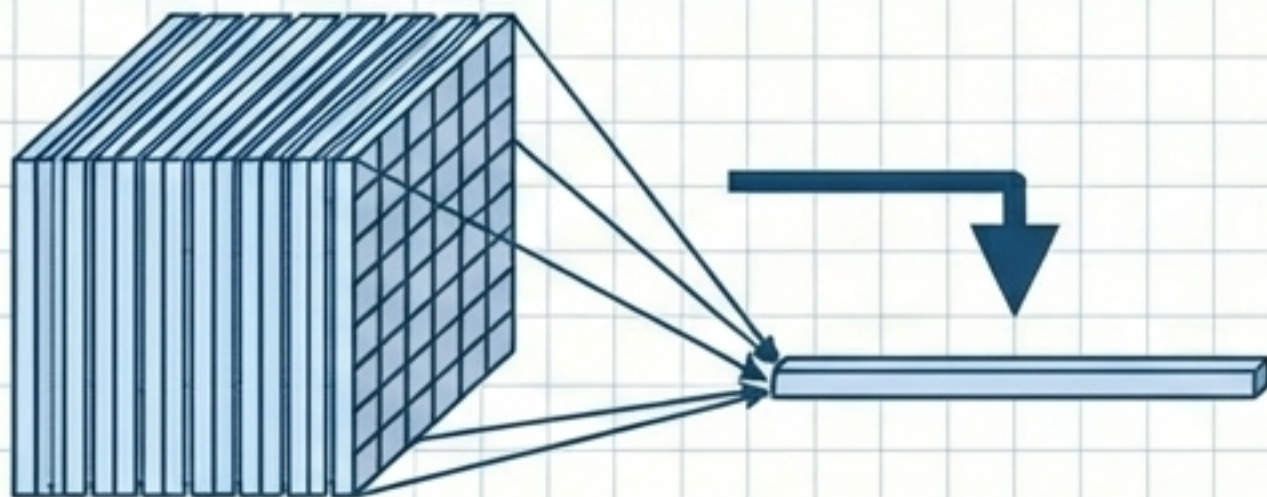
Tải bộ trọng số đã được tối ưu hóa sẵn từ tập dữ liệu ImageNet.

[Rất quan trọng] Cắt bỏ lớp phân loại (Fully Connected) ở đỉnh mạng gốc. Chỉ lấy phần **Trích xuất đặc trưng** (Feature Extractor) để lắp vào bài toán mới.

Thực hành: Thêm các lớp tùy chỉnh

Sau khi tải **base_model**, ta cần thiết kế cái đầu (Head) mới cho kiến trúc.

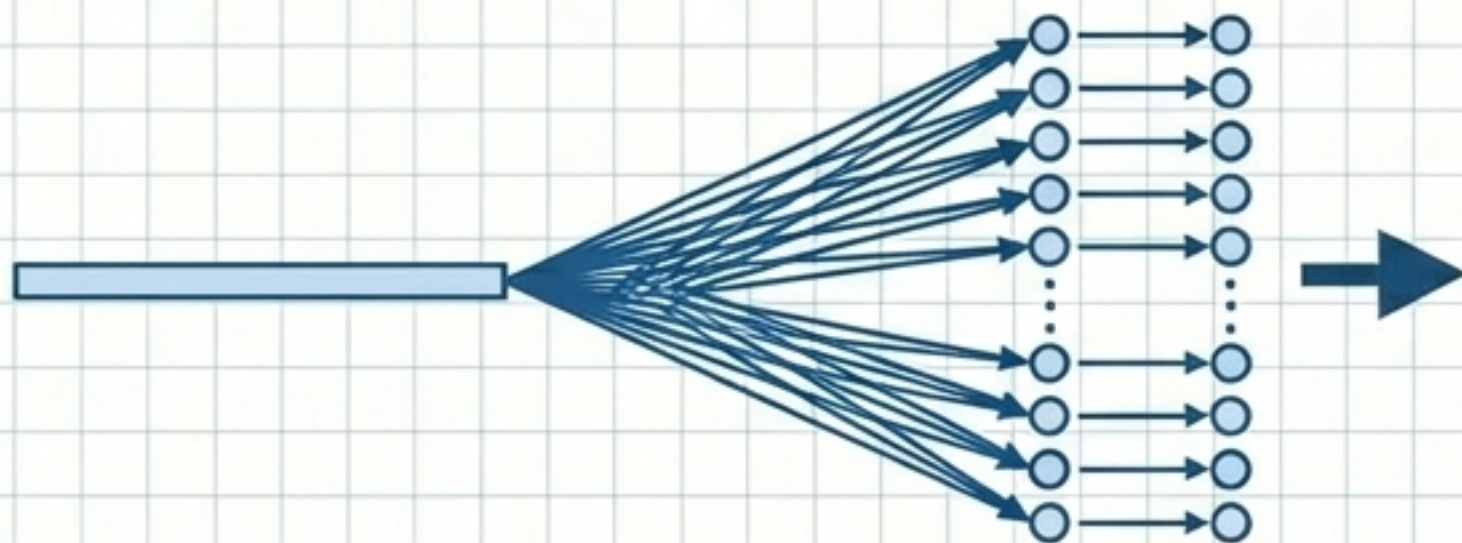
1. Giảm chiều dữ liệu (Global Average Pooling):



```
avg = tf.keras.layers.GlobalAveragePooling  
2D()(base_model.output)
```

Ý nghĩa: Đặt trên đỉnh mô hình cơ sở để tính trung bình các bản đồ đặc trưng, giảm triệt để số lượng tham số, chống Overfitting.

2. Lớp phân loại bài toán mới:



```
class_output = tf.keras.layers.Dense(n_classes,  
activation='softmax')(avg)
```

Ý nghĩa: Lớp Dense với số nơ-ron bằng chính xác số lớp của bài toán hiện tại ($n_classes$), dùng hàm softmax để xuất ra xác suất.

Mở rộng: Học đa tác vụ (Multi-task Learning)

Khái niệm: Rẽ nhánh từ một **Pretrained Model** duy nhất để giải quyết đồng thời nhiều bài toán. (VD: Vừa nhận diện đó là hoa gì, vừa tìm vị trí bông hoa trong ảnh).

Nhánh 1 (Phân loại - Classification)

Hàm mất mát:
categorical_crossentropy

Lớp **Dense**(n_classes,
activation='softmax')

Nhánh 2 (Định vị - Localization)

Hàm mất mát: MSE
(Sai số toàn phương trung bình)

Lớp **Dense**(4) dự đoán 4 tọa độ
(Tâm x, Tâm y, Rộng, Cao)
của Hộp giới hạn (Bounding Box)

base_model

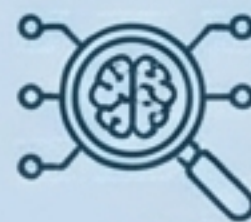
Sự chia sẻ đặc trưng giúp mô hình học nhanh hơn và tiết kiệm tài nguyên tính toán.

Các kho mô hình huấn luyện sẵn (Model Zoos)

Không cần siêu máy tính hay tự thiết kế từ đầu. Hệ sinh thái mã nguồn mở cung cấp miễn phí các kiến trúc State-of-the-Art.

TensorFlow Hub / Keras Applications

Tích hợp sẵn kiến trúc phân loại (VGG, ResNet, EfficientNet) và phát hiện đối tượng (YOLO, SSD, Faster R-CNN) với trọng số chuẩn.



Kho mô hình (Model Zoos)



TorchVision (PyTorch)

Hệ sinh thái đồ sộ tương đương, cập nhật liên tục các bài báo mới nhất, phục vụ phân loại, nhận diện và phân đoạn ảnh (Segmentation).



Hugging Face

Nền tảng chia sẻ mô hình đang nổi lên, không chỉ cho NLP (Transformer) mà còn cho Thị giác máy tính (Computer Vision).

Tổng kết Tiết 2 & Định hướng

- ✓ **Chìa khóa cốt lõi:** Transfer Learning giải bài toán thiếu dữ liệu gán nhãn và tiết kiệm khổng lồ chi phí tính toán phần cứng.
- ✓ **Hai chiến lược chính:** (1) Feature Extraction (Đóng băng lớp dưới) và (2) Fine-Tuning (Mở băng cẩn thận lớp trên với Learning rate cực nhỏ).
- ✓ **Triển khai:** Keras và PyTorch cho phép tải mô hình (ResNet, Xception) cực kỳ nhanh chóng qua tham số `include_top=False`.

Tiếp theo (Tiết 3): Kiến thức về trích xuất đặc trưng bằng CNN & Transfer Learning là nền tảng bắt buộc để chinh phục bài toán phức tạp hơn:
Phát hiện đối tượng (Object Detection - YOLO, Faster R-CNN).